

PLANKAC TECHNICAL REPORT

ENGLISH

ABSTRACT

PlankaC is a substantial executable Plankalkuel-profile implementation in C. It provides a parser, interpreter, typed IR, text bytecode, generated C, generated x86-64 assembly, 8086/DOS-oriented assembly emission, embedding API, host APIs for .plk applications, and a conformance corpus.

The project does not claim to be a complete historical reconstruction of Plankalkuel. Its purpose is narrower and more testable: preserve a source-oriented profile, make it executable, document the implemented semantics, and expose compiler artifacts that can be checked by users.

SOURCE BASIS

The work is oriented around Konrad Zuse's published Plankalkuel material and the later Plankalkuel 2000 implementation report. The bibliography is kept in docs/bibliography.md. Those sources are used for notation, value-bank terminology, plan/procedure structure, two-dimensional rows, structured values, and the chess-modeling tradition.

The project intentionally separates source-derived language features from modern extensions. The 3D geometry profile and graphical host procedures are PlankaC extensions. They are useful, executable examples, not claims about the documented historical core.

ARCHITECTURE

The repository is split into compiler and runtime layers:

- * c/core: source loading, parser support, expression evaluation, runtime API;
- * c/types: marker parsing, type families, compatibility rules;
- * c/notation: linear and PAGE/table two-dimensional notation support;
- * c/analyzer: static checks and structural schema checks;
- * c/values: tagged values, bit packing, and fixed-point helpers;
- * c/ir: typed intermediate representation;
- * c/backends: bytecode, lowering report, generated C, x86-64 ASM, 8086 ASM;
- * c/models: chess board and game-state helpers;
- * c/targets: CLI, Windows GUI, Win16, and DOS hosts;
- * graphics/c: PlankaHost, PlankaGUI, PlankaCube, raster and export layers.

The practical compiler path is:

.plk source -> parser/analyzer -> procedure table -> typed IR/bytecode -> C/ASM/native runner

IMPLEMENTED PROFILE

The implemented profile includes numbered procedures, V/C/Z/R value

banks, type markers with width and scale, guards, assertions, loops, procedure calls, multi-result procedures, indexed values, constants, lists, pairs, sets, relations, records, complex values, bit/fixed helpers, PAGE notation, predicate-like list/relation operations, chess structures, and a focused 3D extension.

The coverage matrix in docs/plankalkuel_coverage.md is the public contract. If a feature is listed as supported, it should have source examples, runtime behavior, documentation, and conformance coverage.

BACKENDS

PlankaC emits several artifacts:

- * text bytecode reloadable by the PlankaC runner;
- * readable typed IR;
- * lowering reports for backend inspection;
- * generated C runners;
- * generated x86-64 ASM runners linked with the runtime on the Windows/MinGW ABI path;
- * 8086/DOS-oriented assembly source.

The current ASM backends are useful engineering artifacts. They do not mean that every compound operation is lowered to bare machine instructions without runtime support. The backend contract is documented explicitly so that future work can deepen the lowering without changing the user-facing source profile. The current executable x86-64 ASM runner targets the Windows/MinGW ABI; Linux CI validates ASM emission structurally and uses generated C/native C for the linked portable native path.

VERIFICATION

The verification entry points are:

```
build.bat
make ci
build/plankac_conformance
```

CI runs Windows and Linux builds. The conformance corpus contains valid, invalid, runtime, backend-equivalence, and Zuse-style example fixtures.

RELEASE DISCIPLINE

Generated executables and compiler outputs are not stored in Git. Tagged releases attach Windows and Linux binaries, source archives, and SHA-256 checksums. Win16 and DOS packages are labeled as experimental platform builds when attached.

REVIEW GOAL

The project is suitable for review by people interested in programming language implementation, computing history, compiler backends, and platform

porting. The desired review is technical: notation fidelity, value/type model boundaries, backend contracts, conformance structure, and documentation clarity.

DEUTSCH

ZUSAMMENFASSUNG

PlankaC ist eine substanzielle ausfuehrbare Plankalkuel-Profil-Implementierung in C. Das Projekt enthaelt Parser, Interpreter, typed IR, Text-Bytecode, generiertes C, generiertes x86-64-ASM, 8086/DOS-nahes ASM, Einbettungs-API, Host-APIs fuer .plk-Anwendungen und einen Conformance-Corpus.

Das Projekt behauptet nicht, eine vollstaendige historische Rekonstruktion des Plankalkuels zu sein. Der Anspruch ist enger und besser pruefbar: ein quellenorientiertes Profil erhalten, ausfuehrbar machen, die implementierten Semantiken dokumentieren und Compiler-Artefakte erzeugen, die Nutzer nachvollziehen koennen.

QUELLENBEZUG

Die Arbeit orientiert sich an Konrad Zuses veroeffentlichtem Plankalkuel-Material und am Plankalkuel-2000-Implementierungsbericht. Die Bibliographie steht in docs/bibliography.md. Diese Quellen geben Richtung fuer Notation, Wertbaenke, Planstruktur, zweidimensionale Zeilen, strukturierte Werte und die Schachmodellierung.

Moderne Erweiterungen werden getrennt benannt. 3D-Geometrie und grafische Host-Profile sind PlankaC-Erweiterungen. Sie sind ausfuehrbare Beispiele, aber keine Behauptung ueber den historischen Sprachkern.

AUFBAU

Die Implementierung ist in Schichten aufgeteilt: Core, Types, Notation, Analyzer, Values, IR, Backends, Models, Targets und Graphics. Der wichtige Pfad ist:

.plk-Quelle -> Parser/Analyzer -> Prozedurtabelle -> typed IR/Bytecode -> C/ASM/native runner

PROFIL

Unterstuetzt werden nummerierte Prozeduren, V/C/Z/R-Baenke, Typmarker, Guards, Assertions, Schleifen, Prozeduraufrufe, mehrere Rueckgabewerte, Indizes, Konstanten, Listen, Paare, Mengen, Relationen, Records, komplexe Werte, Bit-/Fixed-Helfer, PAGE-Notation, praedikatnahe Listen- und Relationsoperationen, Schachstrukturen und eine moderne 3D-Erweiterung.

Die Matrix in docs/plankalkuel_coverage.md ist der oeffentliche Vertrag. Ein als unterstuetzt markiertes Feature soll Quellbeispiele, Laufzeitverhalten, Dokumentation und Conformance-Abdeckung haben.

BACKENDS UND PRUEFUNG

PlankaC erzeugt Bytecode, typed IR, Lowering-Reports, C, x86-64-ASM und 8086/DOS-nahes ASM. Windows und Linux werden in CI gebaut. Die lokalen Einstiege sind build.bat, make ci und der Conformance-Runner.

RELEASE-DISZIPLIN

Binaries und generierte Artefakte liegen nicht im Git-Baum. Releases enthalten separate Windows-/Linux-Bundles, Source-Archive und SHA-256-Pruefsummen. Win16- und DOS-Pakete werden als experimentelle Plattform-Builds bezeichnet, wenn sie veroeffentlicht werden.